

Your software factory already has a data model

The agent can build the model. I still want a human deciding what it means.

My software factory had closed 12,225 work items, and my agent coordinating the flow of work had acknowledged 116 of their outcomes.

Both are plausible answers to “how is our software factory doing?” They are also different facts.

“Closed” is a producer claim: a worker, controller, or cleanup path moved a record to `closed`. “Acknowledged” means an outcome traveled through the coordinator-agent’s ownership and reached an `acknowledged` state.

I wanted to see what a cloud agent would do with distinctions like that. I launched an `Amp` agent in a fresh `Orb` and gave it the whole job: turn my factory’s operational history into a dbt project, model it in Omni, then use Omni in the browser to build and inspect a control-room dashboard.

Agents are totally capable of writing correct SQL, but I wanted to see if the SQL would correctly decide which of those two numbers meant successful work.

I don’t really want my agent deciding that for me in unpredictable ways.

I am happy to let the agent trace the implementation, write the transformations and tests, propose the semantic model, document its reasoning, build the dashboard, and use the application to check what a person will actually see. But, when the evidence supports more than one defensible interpretation, I want the ambiguity surfaced for review rather than collapsed into a metric.

The factory had already modeled itself

My source system runs on `Gas City`, a software-factory SDK I help maintain. I took an August 20 snapshot containing 17,110 work items, 13,035 scheduled-order runs, and 14,544 agent runs.

The data tables I used in my experiment weren’t designed for business intelligence analytics, they exist because the factory needs them to coordinate work. Claims record ownership, events preserve transitions, outcome envelopes track whether produced work made it back to the coordinating agent, and usage journals record model calls.

That gives you plenty to analyze, but the field names do not necessarily mean what an analytics engineer might assume.

Every scheduled order in the snapshot, for example, has `status = 'closed'`. But 2,837 also have a dispatcher result of `failed`, and eight are incomplete. Here, `closed` means the record reached the end of its lifecycle. It does not mean the work succeeded.

Cost has a different problem. I can account for \$373.20 of fully priced model usage and another \$68.60 as a lower bound from partially priced runs, but I cannot reliably join every run to a specific work item. I could still calculate something called “cost per successful task.” The source system does not support that claim.

Time is messy too. Some clients wrote UTC timestamps while others wrote Eastern wall-clock time. Once those values share a column, the timestamp alone cannot tell me which convention produced each row.

The structure is already there. The work is deciding which interpretations of that structure are actually justified.

I gave the agent the whole job

For the experiment, I kept the dbt project, Omni model files, source evidence, and review artifacts together in one monorepo. The Orb received that checkout plus a lightweight dashboard style guide covering KPI hierarchy, layout, labels, and visual treatment.

It built the analytical layer while keeping the distinctions I cared about. `closed` and coordinator acknowledgement remained separate facts. Partially priced usage stayed a lower bound. Questions the evidence could not resolve stayed unresolved.

The agent then carried those decisions into Omni. It created the analytical fields and descriptions, exposed them through an `Omni topic`, opened an isolated model branch with `Branch Mode`, and used Omni’s browser UI to assemble and style the dashboard.

The first screen puts 12,225 claimed closures beside 116 coordinator acknowledgements, makes the 12,109-record gap hard to miss, and shows snapshot provenance and clock uncertainty alongside the operational KPIs. Measures have descriptions, joins have explicit grains, and aggregates can drill back to `work_id`.

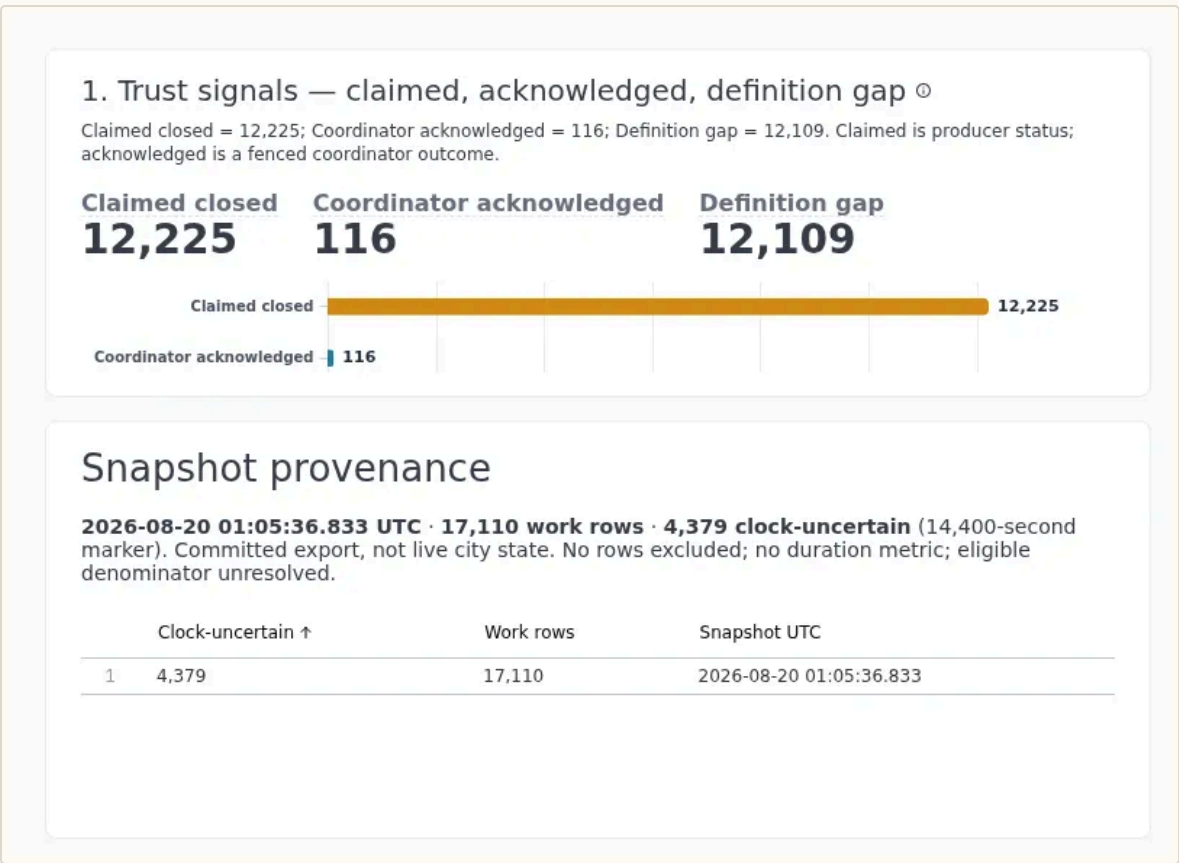
I also wanted the agent working in the application, not stopping at generated model code. A model can be technically correct while the dashboard built from it is misleading. Labels, comparisons, grains, and visual hierarchy all affect what a person thinks the data says.

So the Orb changed the model, built with those fields in Omni, inspected the rendered dashboard, and revised it.

I checked the important values directly against the dbt-built relations in MotherDuck, through Omni’s SQL runner, and through fields exposed by the `Omni topic`. All three returned the same reference values.

By the end, the agent had built 13 dbt models, passed the model tests, created the semantic layer, and assembled seven dashboard tiles.

Figure 01 / Omni control room



The Orb built and styled this in Omni’s browser UI. Large KPIs establish the hierarchy; the zero-based comparison, gap, and provenance keep it honest.

What I changed

The agent still made one decision I would not have approved.

It excluded a work item because its close and start times appeared earlier than its creation time. The exclusion was documented and internally consistent.

But the row was not bad data. The roughly four-hour inversion matched the mixed UTC/Eastern timestamp problem. I wanted the row kept, marked with `clock_uncertainty_seconds = 14400`, and that uncertainty exposed in the dashboard.

I sent the correction back to the agent. It updated the model and the rendered dashboard.

The agent's first choice was reasonable. I still changed it.

That is the part of this workflow I do not want to delegate away. The agent can trace what the code does and propose how that behavior should become a metric. I still need to decide whether I am willing to attach that meaning to the data.

Putting the definition somewhere shared

Approved definitions also need to survive beyond one agent run.

Omni's [dbt integration](#) can carry dbt models and metadata into its semantic layer. The resulting dimensions, measures, joins, descriptions, and permissions can then serve dashboards, human exploration, [Omni's AI](#), and external agents through [Omni MCP](#).

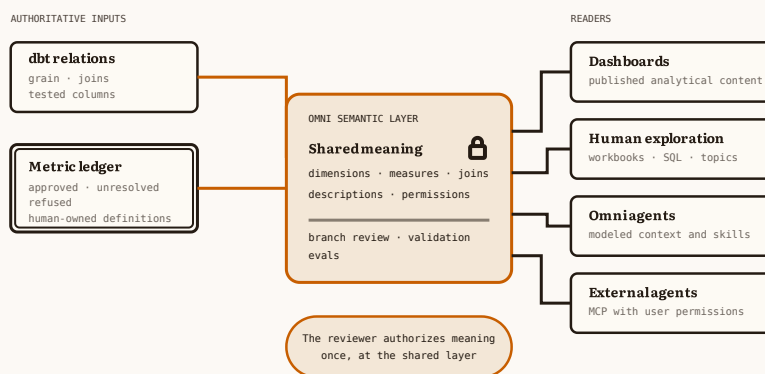
That means the agent can take an ambiguity pretty far before I need to intervene. It can propose a measure, define its SQL and description, expose it through a topic, and build dashboard content against it on a branch.

Then I can review the definition where it will actually be used.

If I approve it and merge it, the interpretation is no longer something one agent inferred for one task. It becomes part of the shared semantic model used by the rest of the system.

Figure 02 / Control substrate

The semantic layer serves every analytical reader



One reviewed definition reaches dashboards, human exploration, Omni's agents, and external agents over MCP.

The agent can trace the source system, build and test the transformations, propose the semantic definitions, assemble the dashboard, inspect it in the browser, and revise it.

What I want to sign off on is the meaning.